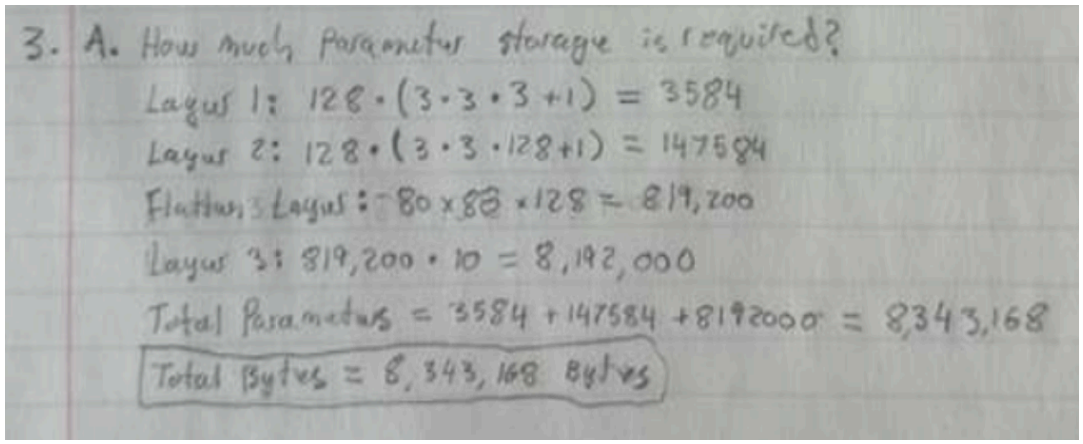
Analysis Problems

1. Consider a fully-connected network with 120 inputs, two hidden layers with 256 units each, and 10 output classes. Assume that all weights and activations are stored as 8-bit values.
    a. Find the total parameter storage required in bytes.
        i. 1. A. Total Parameter Storage required in Bytes (Bias → Input+1)
        Layer 1: 121 × 256 = 30976
        Layer 2: 257 × 256 = 65792
        Layer 3: 257 × 10 = 2570
        Total Parameters = 30976 + 65792 + 2570 = 99338 → 99338 Bytes
        $\boxed{\text{Total Bytes} = 99338 \text{ Bytes}}$
        ii. **Total Bytes = 99338 Bytes**
    b. How many MACs (multiply-and-accumulate) are required to run one inference with this model?
        i. B. How many MAC's are required to run one inference with the model?
        Layer 1: 120 × 256 = 30720
        Layer 2: 256 × 256 = 65536
        Layer 3: 256 × 10 = 2560
        Total MACS = 30720 + 65536 + 2560 = 98816
        $\boxed{\text{Total MACS} = 98816}$
        ii. **Total MACS = 98816 MACS**
    c. How much temporary storage (SRAM) is required to run this model? Remember you'll need to store the outputs from one layer and the outputs from the next layer at the same time. The consecutive pair of layers with the largest combined requirements sets the amount of SRAM required.
        i. C. How much SRAM is required to run this model?
        Layer 1: 120 + 256 = 376
        Layer 2: 256 + 256 = 512
        Layer 3: 256 + 10 = 266
        $\boxed{\text{SRAM Required} = 512 \text{ Bytes}}$
        ii. **SRAM Required = 512 Bytes**

2. Consider a fully-connected network with 1280 inputs, two hidden layers with 512 units each, and 32 output classes. Assume that all weights and activations are stored as 32-bit values.
   a. How much parameter storage is required for this model?
      i.
      2. A: How much Parameter storage is Required? (Bias → Inputs +1)
      Layer 1: 1281 × 512 = 655872
      Layer 2: 513 × 512 = 262656
      Layer 3: 513 × 32 = 16416
      Total Parameters = 655872 + 262656 + 16416 = 934944 Parameters
      Total Bytes: 934944 × 4 Bytes = 3739776 Bytes
      Total Bytes = 3,739,776 Bytes
      ii. **Total Bytes = 3,739,766 Bytes**
   b. Running this model on an 80MHz processor that can compute 1 MAC every 4 cycles, how long will one inference take (answer in ms).
      i.
      2. B. Running this model on an 80MHz processor that can compute 1 MAC every 4 cycles, how long for 1 inference (ms).
      Layer 1: 1280 × 512 = 655360
      Layer 2: 512 × 512 = 262144
      Layer 3: 512 × 32 = 16384
      Total MACs = 655360 + 262144 + 16384 = 933888
      MAC Rate = $\frac{80 \times 10^6}{4} = 20 \times 10^6$ MACs/s
      MACS per inference = 933888 / 20×10^6 = 0.04667s
      MACS Per inference = 0.0467 s → 467 ms
      ii. **MACS Per Inference Time = 467 ms**
   c. How much temporary storage (SRAM) is required to run this model?
      i.
      C. How much SRAM is Required for this model?
      Layer 1: 1280 + 512 = 1792 — 7168
      Layer 2: 512 + 512 = 1024 × 4 Bytes = 4096
      Layer 3: 512 + 32 = 544 — 2176
      SRAM Required = 7168
      ii. **SRAM Required = 7168 Bytes**

3. Consider a standard convolutional network with an 80x80x3 inputs; two hidden layers with 128 channels each, a 3x3 kernel, and "same" padding; followed by a flatten layer and a 10-unit output layer to support 10 classes. Assume that all weights and activations are stored as 8-bit integers.

   a. How much parameter storage is required for this model?

      i. 

3. A. How much Parameter storage is required?

Layer 1: $128 \cdot (3 \cdot 3 \cdot 3 + 1) = 3584$

Layer 2: $128 \cdot (3 \cdot 3 \cdot 128 + 1) = 147584$

Flatten Layer: $80 \times 80 \times 128 = 819{,}200$

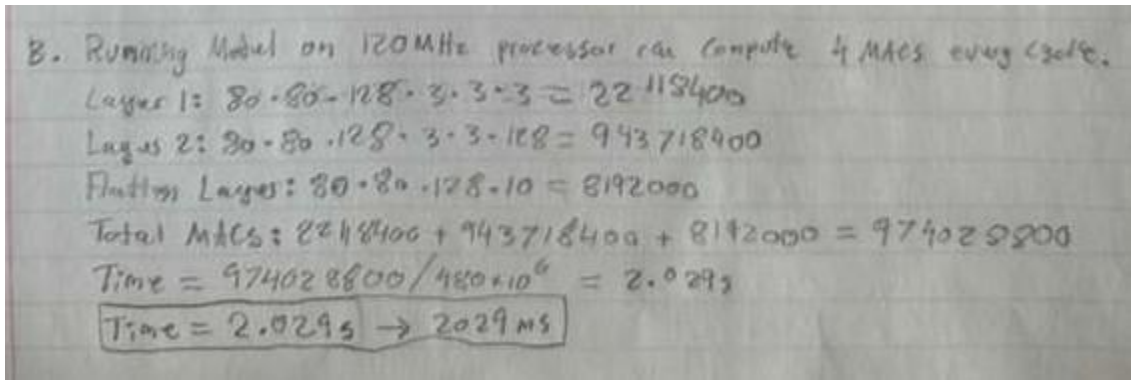
Layer 3: $819{,}200 \cdot 10 = 8{,}192{,}000$

Total Parameters $= 3584 + 147584 + 8192000 = 8{,}343{,}168$

$\boxed{\text{Total Bytes} = 8{,}343{,}168 \text{ Bytes}}$

      ii. **Parameter Storage Required = 8,343,168 Bytes**

   b. Running this model on an 120MHz processor that can compute 4 MACs every cycle, how long will one inference take (answer in ms).

      i. 

B. Running Model on 120MHz processor can compute 4 MACs every cycle.

Layer 1: $80 \cdot 80 \cdot 128 \cdot 3 \cdot 3 \cdot 3 = 22118400$

Layer 2: $80 \cdot 80 \cdot 128 \cdot 3 \cdot 3 \cdot 128 = 943718400$

Flatten Layer: $80 \cdot 80 \cdot 128 \cdot 10 = 8192000$

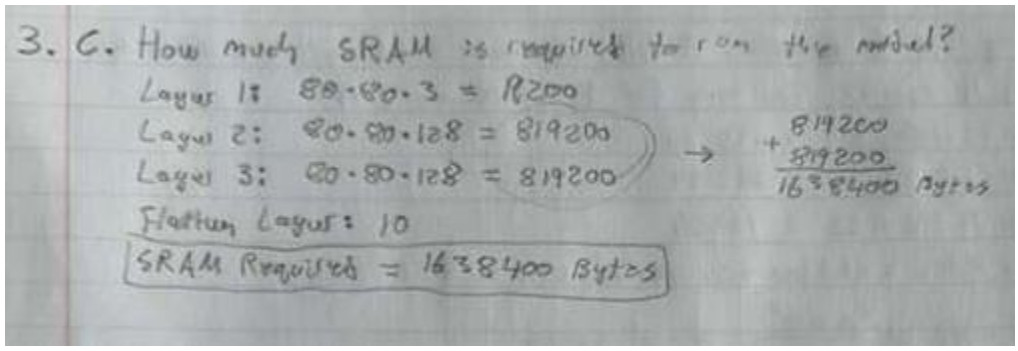
Total MACs: $22118400 + 943718400 + 8192000 = 974028800$

Time $= 974028800 / 480 \times 10^6 = 2.029\text{s}$

$\boxed{\text{Time} = 2.029\text{s} \rightarrow 2029\text{ms}}$

      ii. **MACS Per Inference Time = 2029 ms**

   c. How much temporary storage (SRAM) is required to run this model?

      i. 

3. C. How much SRAM is required to run the model?

Layer 1: $80 \cdot 80 \cdot 3 = 19200$

Layer 2: $80 \cdot 80 \cdot 128 = 819200$

Layer 3: $80 \cdot 80 \cdot 128 = 819200$

$\rightarrow$ $819200 + 819200 = 1638400$ Bytes

Flatten Layer: 10

$\boxed{\text{SRAM Required} = 1638400 \text{ Bytes}}$

      ii. **SRAM Required = 1,638,400 Bytes**

Programming Tasks

1. **Model 1** : What is the training, validation, and test accuracy at the end of training? Did you observe any overfitting? Should the model train for longer, shorter, or about that number of epochs?
   a.
   ```
   1250/1250 [==============================] - 10s 8ms/step - loss: 1.0402 - accuracy: 0.6254 - val_loss: 1.5163 - val_acc
   uracy: 0.5109
   313/313 [==============================] - 1s 4ms/step - loss: 1.4873 - accuracy: 0.5088
   Model 1 Test accuracy: 0.5088000297546387
   ```
   b. Training Accuracy = 62.54%
   c. Validation Accuracy = 51.09%
   d. Testing Accuracy = 50.88%
   e. Model 1 achieved a training accuracy of 62.54%, a validation accuracy of 51.09%, and a test accuracy of 50.88%. Model 1 is overfitting. The model is seen as overfitting because the training accuracy is nearly 11% higher than the validation and testing accuracies. This means that the model is memorizing the training set rather than understanding the information for generalization. The model should be training either the same amount or shorter because making it train longer would not make the validation or testing accuracies better. This is because the fully connected model is not a good model for images, and because the model is memorizing the training data rather than learning from it.
2. **Model 2** : What is the training, validation, and test accuracy at the end of training? Did you observe any overfitting? Should the model train for longer, shorter, or about that number of epochs?
   a.
   ```
   1250/1250 [==============================] - 104s 83ms/step - loss: 0.0461 - accuracy: 0.9834 - val_loss: 1.7481 - val_a
   ccuracy: 0.7208
   313/313 [==============================] - 8s 26ms/step - loss: 1.7544 - accuracy: 0.7167
   Model 2 Test accuracy: 0.71670001745224
   ```
   b. Training Accuracy = 98.34%
   c. Validation Accuracy = 72.08%
   d. Testing Accuracy = 71.67%
   e. Model 2 achieved a training accuracy of 98.34%, a validation accuracy of 72.08%, and a test accuracy of 71.67%. Model 2 is also overfitting, but to a lesser extent than Model 1. The model is seen as overfitting because the training accuracy is much higher than the validation and testing accuracies, indicating that the model is learning the training data extremely well but does not generalize as effectively to unseen data. However, compared to Model 1, both the validation and test accuracies are significantly higher, showing that the convolutional architecture is much better suited for image data. From the training history, validation accuracy improves rapidly in the early epochs and then fluctuates around the low-to-mid 70% range while validation loss steadily increases, suggesting diminishing returns from additional training. As a result, the model should likely be trained for about the same number of epochs or slightly fewer, since training longer would mainly increase training accuracy without

substantially improving validation or test performance. The use of convolutional layers allows the model to learn spatial features, which greatly improves generalization compared to the fully connected model.

3. **Model 2 - Labelling Image** : Does it correctly label the picture?
    a. Dog Image Selected and Scaled to 32x32
    b.
    c. ```
    1/1 [==============================] - 0s 431ms/step
    Predicted class: dog
    ```
    d. Yes, the model labels the image correctly as a dog
4. **Model 3** : For this model, did you observe any overfitting? Should the model train for longer, shorter, or about that number of epochs.
    a. ```
    1250/1250 [==============================] - 71s 57ms/step - loss: 0.1418 - accuracy: 0.9495 - val_loss: 2.3576 - val_ac
    curacy: 0.6062
    313/313 [==============================] - 6s 18ms/step - loss: 2.4347 - accuracy: 0.5974
    Model 3 Test accuracy: 0.5974000096321106
    ```
    b. Training Accuracy = 94.95%
    c. Validation Accuracy = 60.62%
    d. Testing Accuracy = 59.74%
    e. Model 3 achieved a training accuracy of 94.95%, a validation accuracy of 60.62%, and a test accuracy of 59.74%. Model 3 is clearly overfitting, as the training accuracy continues to increase throughout training while the validation and test accuracies plateau and slightly increase. This indicates that the model is learning the training data extremely well but is not improving its ability to generalize to unseen data. From the training history, the validation accuracy increases rapidly during the early epochs and reaches its peak around epochs 7 - 10, after which it remains relatively constant or slightly increases while the validation loss steadily increases. This behavior suggests that the model is over-training. As a result, the model should be trained for fewer epochs, and early stopping would likely improve generalization. Despite the overfitting, Model 3 still outperforms Model 1 in both validation and test accuracy while using depthwise-separable convolutions, which significantly reduce parameter count compared to standard convolutions.
5. **Model Comparison** : How did the three models compare? Consider final accuracy, time per epoch to train, number of epochs needed to reach a given accuracy , overfitting. Build a table to summarize the comparison.

a. The three models show clear differences in performance. Model 1 performed the worst, with about 50.88% test accuracy and strong overfitting, since training accuracy was much higher than validation and test accuracy. Because it is a fully connected network, it does not handle image data well, and training longer would not help. Model 2 performed the best, but had much more parameters than the other two models, and so it took much longer to train. However, Model 2 still reached a 71.67% testing accuracy, much better than the other models. Model 3 falls between Models 1 and 2. It achieved about 59.74% test accuracy while using far fewer parameters than Model 2. However, it still showed overfitting after several epochs. Overall, Model 2 had the best accuracy, Model 3 provided good results for having much less parameters, and Model 1 was mainly a baseline.

b.

Model	Model Type	Training Accuracy	Validation Accuracy	Testing Accuracy
Model 1	Fully Connected	62.54%	51.09%	50.88%
Model 2	Standard Conv2D	98.34%	72.08%	71.67%
Model 3	Separable Conv2D	94.95%	60.62%	59.74%

c.

Model	Number of Parameters	Time per Epoch	Overfitting	Epochs to Reach Good Accuracy
Model 1	430k	Fast	Severe	Did not reach good accuracy
Model 2	620k	Slow	Moderate	5 - 10 Epochs
Model 3	149k	Medium	Moderately High	5 - 7 Epochs

6. **Under 50k Parameters Model** : What did you observe? What model architecture or training decisions made the most difference?

a.

```
1250/1250 [==============================] - 31s 24ms/step - loss: 0.9621 - accuracy: 0.6587 - val_loss: 1.0924 - val_ac
curacy: 0.6205
313/313 [==============================] - 3s 9ms/step - loss: 1.0827 - accuracy: 0.6227
Model 50k Test accuracy: 0.6226999759674072
```

b. The under-50k parameter model achieved a training accuracy of 65.89%, a validation accuracy of about 61.36%, and a test accuracy of 61.11%. This model was designed based on observations from Model 3, which showed that

depthwise-separable convolutions significantly reduce parameter count while still learning and generalizing well. By reducing the number of convolutional layers and channel sizes, the model stayed well under the 50k parameter limit while maintaining reasonable performance. The training accuracy steadily increased across epochs, while validation and test accuracy plateaued around 60-62%, indicating mild overfitting but much less severe than Models 2 and 3. Overall, this model demonstrates that models with less parameters can learn successfully and get decent testing accuracies.